

1. How Different LangChain Components Work Together

We normally create a chain like this:

chain = prompt | model | parser

But internally, how is this possible?

Because these are different components:

- PromptTemplate
- ChatPromptTemplate
- Chat Model
- Output Parser
- Complete Chain

Internally, there is one common concept followed by all these components.

That concept is:

Runnable

All these components follow a common **Runnable-style interface**.

For example, they support methods such as:

invoke()

batch()

stream()

2. Why Is Runnable So Important?

Suppose every component had a completely different API.

Imagine:

prompt.prepare_prompt()

model.ask_ai()

parser.convert_result()

chain.execute_everything()

Then learning and connecting components would become difficult.

Instead, LangChain provides a common Runnable-style interface.

For example:

invoke()

batch()

stream()

Because of this common interface, composition becomes much easier.

Simple logic:

Prompt



Model



Parser

All of them follow the same Runnable concept.

3. What Is a Runnable?

A **Runnable** is a LangChain component that follows a common execution interface.

Simple meaning:

Runnable



Something LangChain can execute

A Runnable:

accepts INPUT



does some work



produces OUTPUT

Basic formula:

INPUT



RUNNABLE



OUTPUT

4. Examples of Runnables

Example 1 — PromptTemplate

Dictionary



PromptTemplate



PromptValue

Example input:

```
{"topic": "Python"}
```

The PromptTemplate converts this information into a formatted prompt.

Example 2 — Chat Model

Input



Chat Model



AIMessage

The model takes the input and generates an AI response.

Example 3 — StrOutputParser

AIMessage



StrOutputParser



String

The parser converts the AI message into a normal Python string.

Example 4 — Complete Chain

Input



Prompt



Model



Parser



Output

The complete chain is also executable like a Runnable.

5. Runnable Formula

Runnable provides common operations such as:

$\text{Runnable} \xrightarrow{\quad} \text{invoke()}$
|
 $\text{Runnable} \xrightarrow{\quad} \text{batch()}$
|
 $\text{Runnable} \xrightarrow{\quad} \text{stream()}$

Simple memory trick:

$\text{invoke()} \rightarrow \text{One Input}$

batch() → Multiple Inputs

stream() → Output Arrives Gradually

6. What Is invoke()?

invoke() executes a Runnable for **one input** and returns **one output**.

Formula:

ONE INPUT

↓

invoke()

↓

ONE OUTPUT

Current LangChain describes invoke() as transforming a single input into an output.

Simple memory trick:

invoke = SINGLE

7. Simple invoke() Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} in very simple English.
```

Give:

- Simple definition
- 3 important points
- One example

```
"""
```

```
)
```

```
model = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
result = chain.invoke(  
    {"topic": "Python"}  
)
```

```
print("\nFinal Answer:")  
print(result)
```

Here:

```
{"topic": "Python"}
```

is one input.

Therefore, we use:

`invoke()`

8. What If We Have Multiple Inputs?

Now imagine we need explanations for:

Python

Java

SQL

LangChain

With `invoke()`, we can certainly write:

```
result1 = chain.invoke(  
    {"topic": "Python"}  
)
```

```
result2 = chain.invoke(  
    {"topic": "Java"}  
)
```

```
result3 = chain.invoke(  
    {"topic": "SQL"}  
)
```

```
result4 = chain.invoke(  
    {"topic": "LangChain"}  
)
```

This works.

But we are repeatedly calling:

```
chain.invoke(...)
```

for every input.

9. Complete Program Using Multiple `invoke()` Calls

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} in very simple English.
```

```
    Give:
```

```
    - Simple definition
```

```
    - 3 important points
```

```
    - One example
```

```
    """
```

```
)
```

```
model = ChatOpenAI(
```

```
    model="gpt-5.6-luna",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
result1 = chain.invoke(  
    {"topic": "Python"}  
)
```

```
result2 = chain.invoke(  
    {"topic": "Java"}  
)
```

```
result3 = chain.invoke(  
    {"topic": "SQL"}  
)
```

```
result4 = chain.invoke(  
    {"topic": "LangChain"}  
)
```

```
print("Information about Python")  
print(result1)
```

```
print("=====")
```

```
print("Information about Java")  
print(result2)
```

```
print("=====")
```

```
print("Information about SQL")
```

```
print(result3)
```

```
print("=====")
```

```
print("Information about LangChain")
```

```
print(result4)
```

```
print("=====")
```

This program is valid.

But when we have many inputs, writing many `invoke()` calls is inconvenient.

To overcome this difficulty, `Runnable` provides the:

```
batch()
```

method.

10. What Is `batch()`?

`batch()` executes a `Runnable` for **multiple inputs** and returns **multiple outputs**.

Formula:

MULTIPLE INPUTS

↓

```
batch()
```

↓

MULTIPLE OUTPUTS

For beginners, remember:

`invoke()` = one input

batch() = many inputs

Important

The input to batch() is a list of inputs.

The result returned by batch() is also a list.

Example:

```
inputs = [  
    {"topic": "Python"},  
    {"topic": "Java"},  
    {"topic": "SQL"}  
]
```

Then:

```
results = chain.batch(inputs)
```

11. Simple batch() Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(  
    "Explain {topic} in one simple sentence."  
)
```

```
model = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
inputs = [  
    {"topic": "Python"},  
    {"topic": "Java"},  
    {"topic": "SQL"}  
]
```

```
results = chain.batch(inputs)
```

```
print(type(results)) # list
```

```
for result in results:
```

```
    print(result)
```

```
    print("-----")
```

Conceptually:

Input 1 → Python

Input 2 → Java

Input 3 → SQL

↓

batch()

↓

Output 1

Output 2

Output 3

12. How Does batch() Work?

Current LangChain's default batch() implementation executes invoke() across a list of inputs.

For suitable I/O-bound Runnables, it may use parallel/concurrent execution.

Conceptually:

Input 1 —→ invoke() —→ Output 1

Input 2 —→ invoke() —→ Output 2

Input 3 —→ invoke() —→ Output 3

For beginners, the most important point is:

invoke() → single input

batch() → multiple inputs

13. Does batch() Mean One Giant Prompt?

No.

Very important.

Suppose:

```
inputs = [  
    {"topic": "Python"},  
    {"topic": "Java"},  
    {"topic": "SQL"}  
]
```

batch() does **not** simply create one giant prompt like:

Explain Python, Java and SQL together.

Instead, conceptually these are separate inputs processed through the same Runnable.

Input 1 → Chain → Output 1

Input 2 → Chain → Output 2

Input 3 → Chain → Output 3

So:

Python → separate input

Java → separate input

SQL → separate input

14. Python for Loop vs batch()

Students may ask:

Sir, why can't we simply use a Python for loop?

Yes, we can.

Example:

for item in inputs:

```
    result = chain.invoke(item)
```

```
    print(result)
```

This is completely valid.

But Runnable already provides its built-in batch interface:

```
results = chain.batch(inputs)
```

The current default LangChain implementation can run `invoke()` calls concurrently using a thread pool for appropriate I/O-bound work.

So we have two approaches.

Using Python Loop

for item in inputs:

```
    result = chain.invoke(item)
```

Using Runnable `batch()`

```
results = chain.batch(inputs)
```

`batch()` gives us the standard Runnable interface for processing multiple inputs.

15. Do Not Say "`batch()` Is Always Faster"

Very important.

Do not tell students:

`batch()` is always faster.

That statement is too strong.